

1/7/2026

* matrix operations

matrix operations are mathematical operations performed on matrices.

Syntax

```
import numpy as np
```

```
A = np.array([ [1,2], [3,4] ])
```

```
B = np.array([ [5,6], [7,8] ])
```

Ex:-

Addition $\Rightarrow A + B$

Sub $\Rightarrow A - B$

multi $\Rightarrow (A, B)$ or $(A @ B)$

matrix transpose $\Rightarrow A.T$

* functions

~~np~~ numpy provides many built-in functions for performing mathematical and statistical operations on arrays.

Syntax:-

```
np.function_name(array)
```

Ex:-

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40])
```

np.sum(arr)	# Sum of elements
np.mean(arr)	# Average
np.max(arr)	# max value
np.min(arr)	# min value
np.sqrt(arr)	# square root of element

* Linear Algebra (NumPy)

The `numpy.linalg` module provide function for solving linear algebra problems such as finding determinants and solving systems of equations.

Syntax `np.linalg.function_name(matrix)`

Ex:-

```
import numpy as np
```

```
A = np.array([[1, 2],  
               [3, 4]])
```

- ~~np.det~~
- np.linalg.det(A) # Determinant
- np.linalg.inv(A) # Inverse
- np.linalg.eig(A) # Eigenvalues
- solve $AX = B$
B = np.array([5, 6])

* Saving Numpy arrays

Numpy allows you to save arrays to files and load them later using .npy or .npz formats

Syntax

```
np.save ("filename.npy", array)
```

```
np.load ("filename.npy")
```

Ex:-

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4])
```

```
- np.save ("data.npy", arr) # save
```

```
- np.load ("
```

```
- load_arr = np.load ("data.npy") # load
```

```
Print (load_arr)
```